

TRD

Technical Requirements Document (TRD)

AWS S3-Based Data Processing Pipeline

-

TR-INGEST-001

Related Functional Requirement(s)

FR-INGEST-001

Objective

Implement S3 data ingestion for customer and order CSV files using AWS Glue Spark job.

Target Cluster Configuration

- Service: AWS Glue ETL Job
- Worker Type: G.1X or G.2X
- Number of Workers: 2-5 (based on data volume)
- Glue Version: 3.0 or higher (includes Spark 3.1, Python 3.7)
- Job Type: Spark

Source Data Details

- Customer Data:
 - Location: `s3://adif-sdlc/sdlc\_wizard/customerdata/`
 - Format: CSV
 - Schema:
 - CustId (String)
 - Name (String)
 - EmailId (String)
 - Region (String)
 - Encoding: UTF-8
- Order Data:
 - Location: `s3://adif-sdlc/sdlc\_wizard/orderdata/`
 - Format: CSV
 - Schema:
 - OrderId (String)

- ItemName (String)
- PricePerUnit (Decimal/Double)
- Qty (Integer)
- Date (String/Date)
- CustId (String)
- Encoding: UTF-8

Target Data Details

- Target: In-memory Spark DataFrames
- Output: customer_df, order_df

Job Flow / Pipeline Stages

1. Initialize Glue Spark context
2. Read customer CSV files from S3 using `spark.read.csv()` with header=true, inferSchema=true
3. Read order CSV files from S3 using `spark.read.csv()` with header=true, inferSchema=true
4. Validate schema conformance for both datasets
5. Pass DataFrames to cleaning stage

Data Transformations / Business Logic

- Parse CSV with comma delimiter
- Infer or explicitly define schema matching source structure
- No transformations applied at ingestion stage

Error Handling and Logging

- Log S3 read errors to CloudWatch
- Validate file existence before read operation
- Capture schema mismatch exceptions
- Log record counts post-ingestion

Assumptions

- CSV files use comma delimiter without embedded commas requiring special handling
- Header row present in all CSV files
- Files are well-formed and parseable by Spark CSV reader
- IAM role attached to Glue job has `s3:GetObject` permission on source buckets
-

TR-CLEAN-001

Related Functional Requirement(s)

FR-CLEAN-001, FR-CLEAN-002

Objective

Remove null values and duplicate records from customer and order datasets.

Target Cluster Configuration

- Same Glue job as TR-INGEST-001 (continuation of pipeline)

Source Data Details

- Input: customer_df, order_df (from TR-INGEST-001)

Target Data Details

- Output: customer_clean_df, order_clean_df

Job Flow / Pipeline Stages

1. Apply null removal to customer_df
2. Apply duplicate removal to customer_df
3. Apply null removal to order_df
4. Apply duplicate removal to order_df
5. Pass cleaned DataFrames to catalog registration stage

Data Transformations / Business Logic

- Null Removal:
 - Filter out rows where any column contains string "Null" (case-insensitive match)
 - Filter out rows where any column is NULL using `df.na.drop()`
- Duplicate Removal:
 - Apply `df.dropDuplicates()` on all columns
 - Retain first occurrence of duplicate rows

Error Handling and Logging

- Log count of null records removed per dataset
- Log count of duplicate records removed per dataset
- Log final cleaned record counts to CloudWatch

Assumptions

- "Null" string matching is case-insensitive
- Duplicate detection uses exact match across all columns
- No business key prioritization for deduplication
-

TR-CATALOG-001

Related Functional Requirement(s)

FR-CATALOG-001

Objective

Register cleaned customer and order datasets as Glue Data Catalog tables.

Target Cluster Configuration

- Same Glue job as TR-INGEST-001 (continuation of pipeline)

Source Data Details

- Input: customer_clean_df, order_clean_df

Target Data Details

- Customer Table:
 - Database: gen_ai_poc_databricksco
 - Table Name: sdlc_wizard_customer
 - Schema:
 - CustId (string)
 - Name (string)
 - EmailId (string)
 - Region (string)
 - Format: Parquet (recommended for Athena performance)
 - Location: `s3://adif-sdlc/catalog/sdlc\_wizard\_customer/`
- Order Table:
 - Database: gen_ai_poc_databricksco
 - Table Name: sdlc_wizard_order
 - Schema:
 - OrderId (string)
 - ItemName (string)
 - PricePerUnit (double)
 - Qty (int)
 - Date (string)
 - CustId (string)
 - Format: Parquet
 - Location: `s3://adif-sdlc/catalog/sdlc\_wizard\_order/`

Job Flow / Pipeline Stages

1. Write customer_clean_df to S3 in Parquet format
2. Create/update Glue catalog table `sdlc\_wizard\_customer` using `glueContext.write\_dynamic\_frame.from\_catalog()`
3. Write order_clean_df to S3 in Parquet format
4. Create/update Glue catalog table `sdlc\_wizard\_order`
5. Verify table registration via Glue API

Data Transformations / Business Logic

- Convert Spark DataFrames to Glue DynamicFrames
- Write data in Parquet format for optimized Athena queries
- Update catalog metadata with schema and location

Error Handling and Logging

- Log catalog write errors to CloudWatch
- Validate table creation/update success
- Log S3 write completion and record counts

Assumptions

- Glue database `gen\_ai\_poc\_databrickscoe` exists or is created via separate setup script
- IAM role has `glue:CreateTable`, `glue:UpdateTable` permissions
- S3 locations for catalog tables are writable
-

TR-SCD-001

Related Functional Requirement(s)

FR-SCD-001

Objective

Implement SCD Type 2 for order summary using Apache Hudi on S3.

Target Cluster Configuration

- Service: AWS Glue ETL Job
- Worker Type: G.2X (Hudi requires more memory)
- Number of Workers: 5-10
- Glue Version: 3.0 or higher
- Additional JARs: Hudi Spark bundle (included in Glue 3.0+)
- Job Parameters:

- `--enable-glue-datacatalog` : true`
- `--datalake-formats` : hudi`

Source Data Details

- Input: `sdlc_wizard_customer`, `sdlc_wizard_order` (from Glue catalog)

Target Data Details

- Order Summary Table:
- Database: `gen_ai_poc_databrickscoe`
- Table Name: `ordersummary`
- Location: `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/``
- Format: Hudi (Copy-on-Write)
- Schema:
- `CustId` (string)
- `Name` (string)
- `EmailId` (string)
- `Region` (string)
- `OrderId` (string)
- `ItemName` (string)
- `PricePerUnit` (double)
- `Qty` (int)
- `Date` (string)
- `IsActive` (boolean)
- `StartDate` (timestamp)
- `EndDate` (timestamp, nullable)
- `OpTs` (timestamp)
- Hudi Configuration:
- Record Key: `OrderId`
- Precombine Field: `OpTs`
- Table Type: `COPY_ON_WRITE`
- Partition Path: `Date` (optional, based on query patterns)

Job Flow / Pipeline Stages

1. Read customer and order tables from Glue catalog
2. Perform inner join on `CustId`
3. Add SCD2 columns: `IsActive=true`, `StartDate=current_timestamp()`, `EndDate=null`, `OpTs=current_timestamp()`

4. Read existing ordersummary Hudi table (if exists)
5. Identify new and changed records via comparison
6. For changed records: mark existing as inactive (IsActive=false, EndDate=current_timestamp())
7. Insert new active records
8. Write to Hudi table using upsert operation
9. Update Glue catalog with Hudi table metadata

Data Transformations / Business Logic

- Join Logic:

...

```
joined_df = customer_df.join(order_df, "CustId", "inner")
```

...

- SCD2 Logic:
- Compare joined records with existing ordersummary on all business columns (excluding SCD2 metadata)
- New records: insert with IsActive=true, StartDate=now, EndDate=null, OpTs=now
- Changed records:
- Update existing row: set IsActive=false, EndDate=now
- Insert new row: IsActive=true, StartDate=now, EndDate=null, OpTs=now
- Unchanged records: no action
- Hudi Write Options:

...

```
hudi_options = {  
'hoodie.table.name': 'ordersummary',  
'hoodie.datasource.write.recordkey.field': 'OrderId',  
'hoodie.datasource.write.precombine.field': 'OpTs',  
'hoodie.datasource.write.operation': 'upsert',  
'hoodie.datasource.write.table.type': 'COPY_ON_WRITE',  
'hoodie.datasource.hive_sync.enable': 'true',  
'hoodie.datasource.hive_sync.database': 'gen_ai_poc_databrickscoe',  
'hoodie.datasource.hive_sync.table': 'ordersummary',  
'hoodie.datasource.hive_sync.use_jdbc': 'false',  
'hoodie.datasource.hive_sync.partition_extractor_class':  
'org.apache.hudi.hive.MultiPartKeyValueExtractor'  
}
```

...

Error Handling and Logging

- Log join operation record counts
- Log SCD2 operation metrics (inserts, updates)
- Capture Hudi write failures and log to CloudWatch
- Validate Hudi table commit success
- Log catalog sync status

Assumptions

- Change detection compares all business columns (CustId through Date)
- OpTs serves as tie-breaker for Hudi precombine logic
- Timezone for timestamps is UTC
- Initial load treats all records as new (IsActive=true)
-

TR-SCD-002

Related Functional Requirement(s)

FR-SCD-002

Objective

Implement incremental SCD2 updates triggered by customer data changes.

Target Cluster Configuration

- Same as TR-SCD-001

Source Data Details

- Input:
- Incremental customer updates from `s3://adif-sdlc/sdlc\_wizard/customerdata/`
- Existing ordersummary Hudi table

Target Data Details

- Output: Updated ordersummary Hudi table

Job Flow / Pipeline Stages

1. Identify changed/new customer records since last processing
2. Extract CustId list from changed customers
3. Filter orders linked to changed customers
4. Read affected ordersummary records using CustId filter

5. Apply SCD2 logic to affected records only
6. Perform Hudi upsert operation
7. Update Glue catalog metadata

Data Transformations / Business Logic

- Change Detection:
 - Compare incoming customer records with sdhc_wizard_customer table
 - Identify changes via hash comparison or timestamp-based filtering
 - Extract distinct CustId values for changed customers
- Incremental SCD2:
 - Filter ordersummary where CustId in changed_customer_ids and IsActive=true
 - Apply same SCD2 logic as TR-SCD-001 to filtered subset
 - Use Hudi upsert to update only affected records

Error Handling and Logging

- Log count of changed customers detected
- Log count of affected ordersummary records
- Log incremental update metrics
- Capture and log any Hudi commit failures

Assumptions

- Customer changes detected via comparison with previous snapshot or CDC mechanism
- Incremental processing triggered on schedule (e.g., daily batch)
- Only active ordersummary records are candidates for update
-

TR-AGG-001

Related Functional Requirement(s)

FR-AGG-001

Objective

Generate customer aggregate spend table from active order summary records.

Target Cluster Configuration

- Service: AWS Glue ETL Job
- Worker Type: G.1X
- Number of Workers: 2-5
- Glue Version: 3.0 or higher

Source Data Details

- Input: ordersummary table (from Glue catalog)
- Filter: IsActive = true

Target Data Details

- Customer Aggregate Spend Table:
- Database: gen_ai_poc_databricksco
- Table Name: customeraggregatespend
- Location: `s3://adif-sdlc/analytics/customeraggregatespend/`
- Format: Parquet
- Schema:
- Name (string)
- TotalAmount (double)
- Date (string)

Job Flow / Pipeline Stages

1. Read ordersummary table from Glue catalog
2. Filter for IsActive = true
3. Calculate TotalAmount per customer
4. Write aggregated results to S3 in Parquet format
5. Register/update customeraggregatespend table in Glue catalog

Data Transformations / Business Logic

- Aggregation SQL Logic:

...

SELECT

Name,

SUM(PricePerUnit Qty) AS TotalAmount,

Date

FROM ordersummary

WHERE IsActive = true

GROUP BY Name, Date

...

- Spark DataFrame Implementation:

...

```
agg_df = ordersummary_df.filter(col("IsActive") == True) \
```

```
.withColumn("Amount", col("PricePerUnit") col("Qty")) \
.groupBy("Name", "Date") \
.agg(sum("Amount").alias("TotalAmount"))
```
```

### ### Error Handling and Logging

- Log count of active records processed
- Log aggregation result record count
- Capture write failures to S3
- Log catalog registration status

### ### Assumptions

- TotalAmount calculated as sum of (PricePerUnit Qty) per customer
- Date field represents order date or aggregation execution date
- Only active ordersummary records included in calculation
- Aggregation runs as full refresh (not incremental)
- 

## ## TR-ARCH-001

### ### Related Functional Requirement(s)

FR-ARCH-001

### ### Objective

Define AWS architecture components and integration points for the pipeline.

### ### Target Cluster Configuration

N/A (Architecture-level requirement)

### ### Source Data Details

N/A

### ### Target Data Details

N/A

### ### Job Flow / Pipeline Stages

- Component Integration:
  1. S3: Storage for all data layers (source, catalog, curated, analytics)
  2. AWS Glue: ETL processing engine (Spark + Hudi)
  3. Glue Data Catalog: Metadata repository for all tables

4. Amazon Athena: Query engine for ad-hoc analysis
5. CloudWatch: Logging and monitoring for Glue jobs
6. Lake Formation (Optional): Data governance and access control

- IAM Roles and Policies:
- Glue service role with permissions:
  - `s3:GetObject`, `s3:PutObject` on all pipeline buckets
  - `glue:CreateTable`, `glue:UpdateTable`, `glue:GetTable`
  - `logs:CreateLogGroup`, `logs:CreateLogStream`, `logs:PutLogEvents`
  - `lakeformation:GetDataAccess` (if Lake Formation enabled)
- Networking:
  - Glue jobs run in AWS-managed VPC (default) or customer VPC if required
  - S3 VPC endpoints recommended for private connectivity

### Data Transformations / Business Logic

N/A

### Error Handling and Logging

- CloudWatch Configuration:
  - Log Group: `/aws-glue/jobs/`
  - Log Streams: Per job execution
  - Retention: 30 days (configurable)
  - Metrics: Job duration, DPU usage, record counts
- Monitoring:
  - CloudWatch alarms for job failures
  - SNS notifications for critical errors
  - Glue job bookmarks for incremental processing tracking

### Assumptions

- All services deployed in same AWS region (e.g., us-east-1)
- S3 server-side encryption enabled (SSE-S3 or SSE-KMS)
- Lake Formation optional; not required for initial implementation
- No VPC isolation required; Glue uses AWS-managed networking
- CloudWatch log retention follows organizational policy (default 30 days)
- No real-time streaming; all processing is batch-based
- Data retention: indefinite for SCD2 historical records

- No data masking or column-level encryption beyond S3 encryption
- Athena query optimization (partitioning) deferred to post-implementation tuning
- 

## ## GLOBAL ASSUMPTIONS

1. Data Volume: Moderate data volumes suitable for Glue standard worker types (G.1X/G.2X)
2. Processing Frequency: Batch processing (daily or on-demand); no real-time requirements
3. Data Quality: Beyond null and duplicate removal, no additional validation rules implemented
4. Error Handling: Glue job retries configured at job level (default: 0 retries, configurable to 1-3)
5. Concurrency: Single job execution at a time; no parallel pipeline runs
6. Schema Evolution: Not addressed; schema changes require manual intervention
7. Data Lineage: Tracked via Glue job execution logs and CloudWatch
8. Cost Optimization: No specific optimization requirements; standard Glue pricing applies
9. Security: Standard AWS security best practices; no custom encryption or tokenization
10. Testing: Unit testing and integration testing deferred to implementation phase
11. Disaster Recovery: Relies on S3 versioning and cross-region replication (if configured)
12. Performance SLAs: No specific SLAs defined; processing time depends on data volume